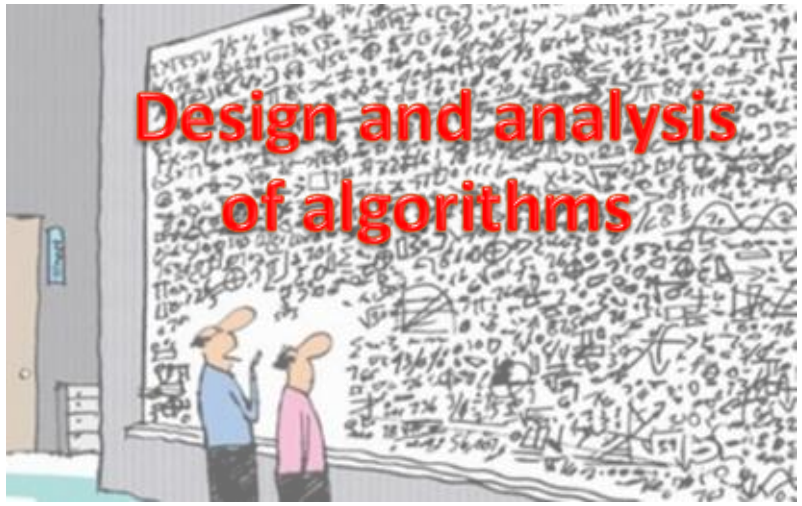


Analysis of graphs

- Depth-First Search
  - Cycle detection
  - Topological sort

Objective

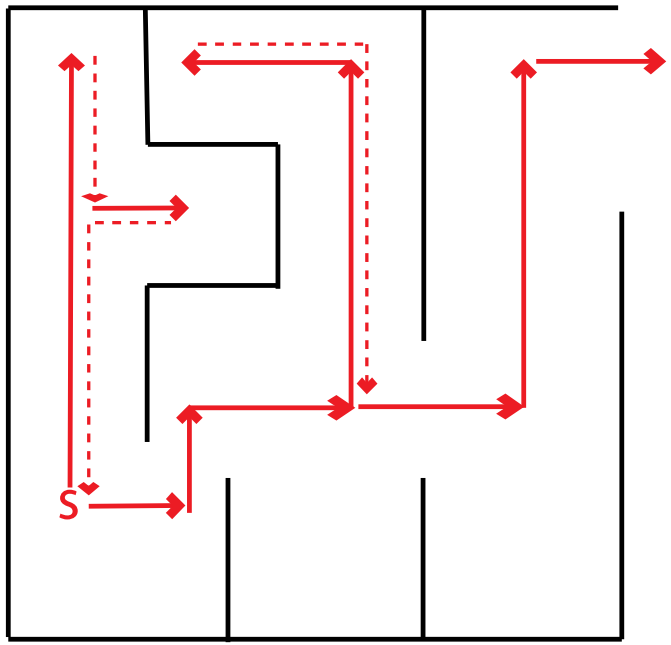
- Define the depth-first search and the types of edges.
- Cycle detection.
- Topological sort.
- DPV Chapter 3.

Graph searches

- We want to get from current Rubik state to “solved” state
  - How do we explore the graph?
- We are in a maze given by a graph.
  - How do we walk on the graph to find an exit?
- May use different procedures!! -> BFS, DFS.
- https://www.cs.usfca.edu/~galles/visualization/BFS.html
- https://www.cs.usfca.edu/~galles/visualization/DFS.html

Depth-first search

- Depth-first search is a surprisingly versatile linear-time procedure that **reveals a wealth of information** about a graph.
- The most basic question it addresses:
  - What parts of the graph are reachable from a given vertex?
- Exploring a graph = like exploring a maze.
  - From current vertex, move to another.
  - Until you get stuck.
  - Then backtrack till you find a new place to explore.
  - You may use the “left-hand” rule.



DFS algorithm

- Find the nodes reachable from a given node v.

from DPV

```
procedure explore(G, v)
Input:    G = (V, E) is a graph; v ∈ V
Output:   visited(u) is set to true for all nodes u reachable from v

visited(v) = true
previsit(v)
for each edge (v, u) ∈ E:
    if not visited(u):  explore(u)
postvisit(v)
```

```
procedure dfs(G)

for all v ∈ V:
    visited(v) = false

for all v ∈ V:
    if not visited(v):  explore(v)
```

```
procedure previsit(v)
pre[v] = clock
clock = clock + 1
```

```
procedure postvisit(v)
post[v] = clock
clock = clock + 1
```

DFS algorithm (CLRS):

With colors and timing

```
DFS(G)
1  for each vertex u ∈ G.V
2      u.color = WHITE
3      u.π = NIL
4  time = 0
5  for each vertex u ∈ G.V
6      if u.color == WHITE
7          DFS-VISIT(G, u)

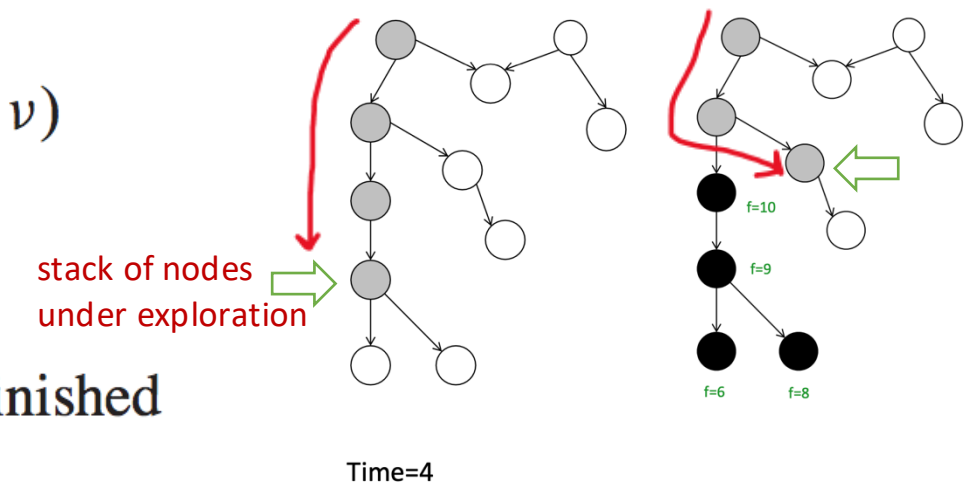
DFS-VISIT(G, u)
1  time = time + 1                  // white vertex u has just been discovered
2  u.d = time
3  u.color = GRAY
4  for each v ∈ G.Adj[u]            // explore edge (u, v)
5      if v.color == WHITE
6          v.π = u
7          DFS-VISIT(G, v)
8  u.color = BLACK                  // blacken u; it is finished
9  time = time + 1
10 u.f = time
```

in DPV book this "pre" time

in DPV book this "post" time

timestamps

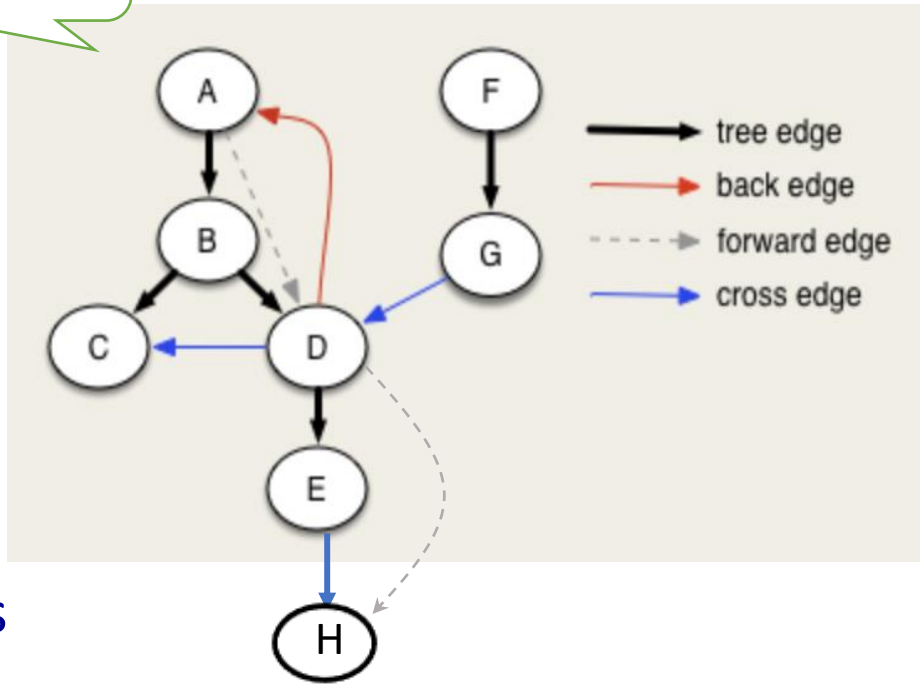
What we do more here:
- use 3 colors: white (new), gray (on DFS stack), black (finished),
- remember parent node.



Time=4

Types of edges

In the DFS tree we show the tree edges



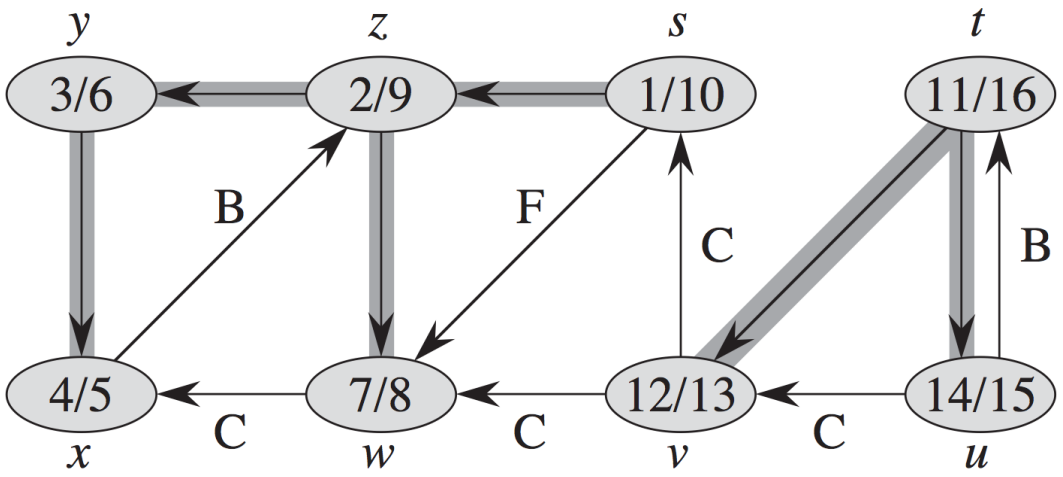
- We consider the edge (u, v) at the time we explore it:

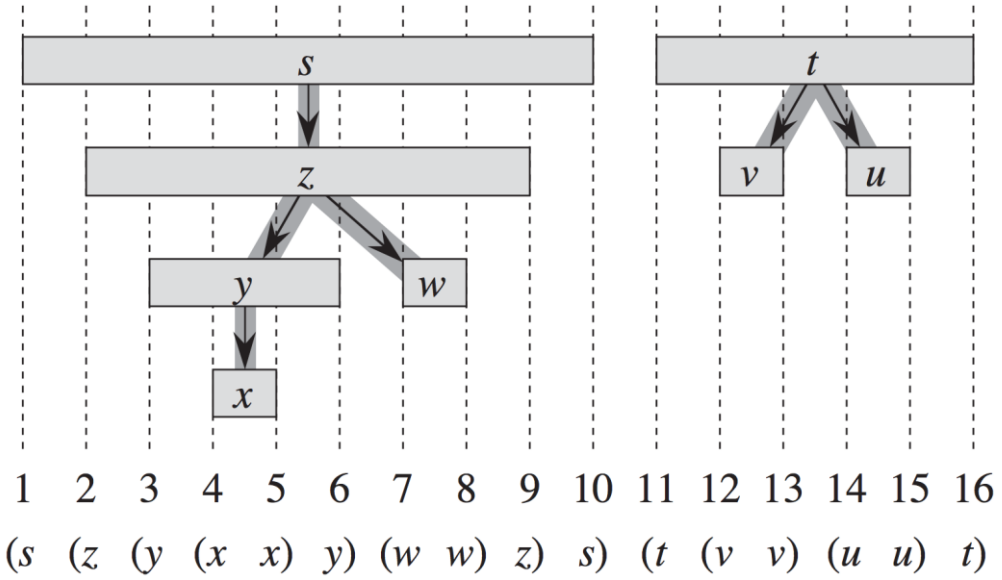
1. If v is visited for the first time as we traverse the edge (u, v), then the edge is a tree edge
2. Else, v has already been visited:
   - If v is an ancestor of u, then edge (u, v) is a back edge.
   - (b) Else, if v is a descendant of u, then edge (u, v) is a forward edge.
   - (c) Else, if v is neither an ancestor or descendant of u, then edge (u, v) is a cross edge. These type of edges result when u and v are reached by different branches.

ancestor and descendant: properties checked using the parent pointers $v.\pi$

Observe that the type of an edge is defined for a specific execution of the DFS.

Properties of DFS





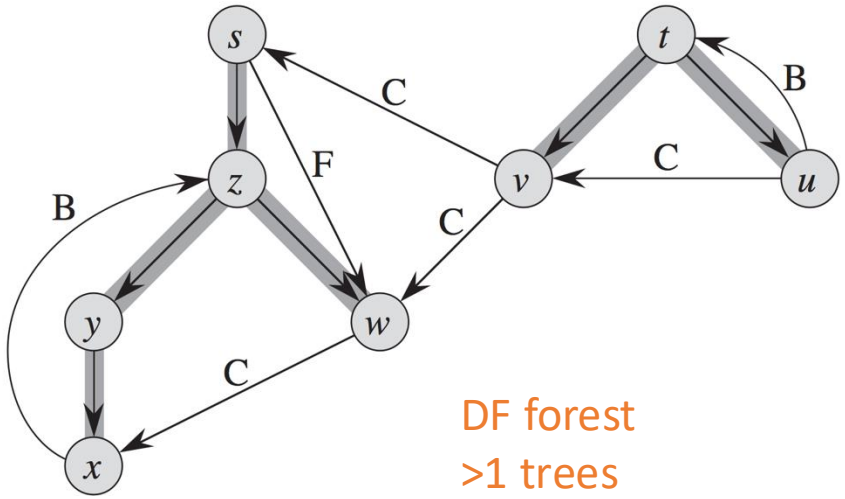
Type of edge $u \to v$

back: $v.d < u.d < u.f < v.f$

forward: $u.d < v.d < v.f < u.f$

cross: $v.d < v.f < u.d < u.f$

tree: first time v is visited

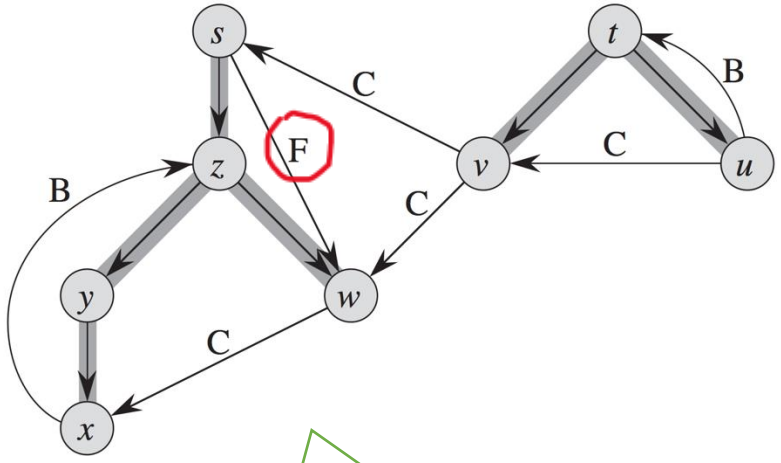


DF forest
>1 trees

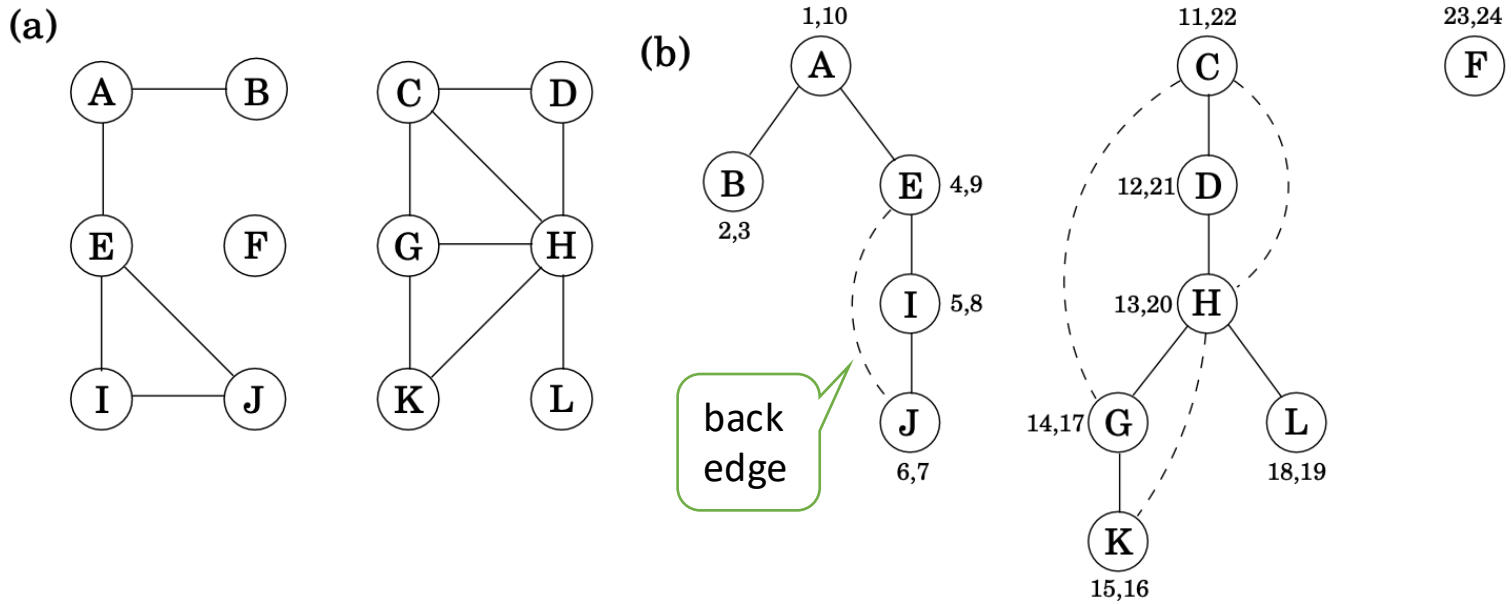
We use the timing information to determine the type of edges

Properties of DFS for undirected graphs

G undirected: only tree and back edges



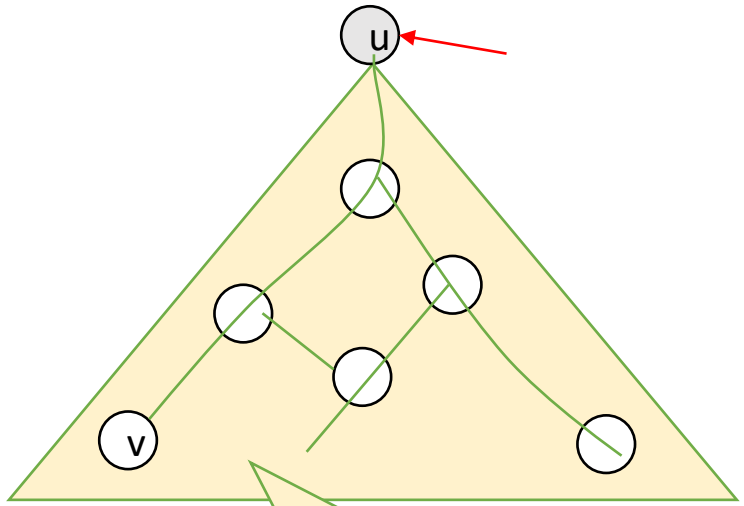
can we have **forward** and **cross** edges if *G* is undirected?



Example: edge F cannot be a forward edge if the graph is undirected. It would have been considered when we explore node w, and it would be a back edge.

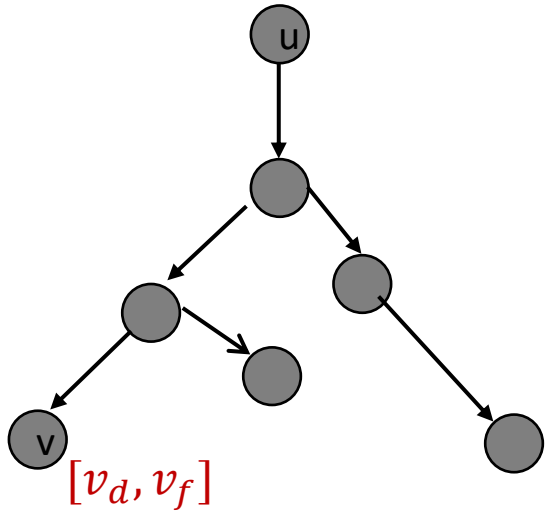
"White path **theorem**" (CLRS Theorem 22.9, see proof).

- Let W_u be the set of white nodes reachable from u at the time of its discovery.
- Then: they will all be connected to u with a path of tree edges in the DFS tree,
  $\Leftrightarrow$ at the time of their exploration u is grey $\Leftrightarrow$ they will be finished before u
  $\Leftrightarrow u_d < v_d < v_f < u_f$.

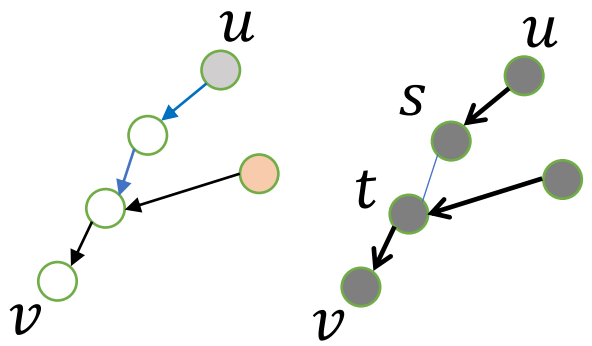


all **white** nodes *reachable* from u at the time of its discovery

$u_d < v_d < v_f < u_f$



otherwise, it would be a descendent of u



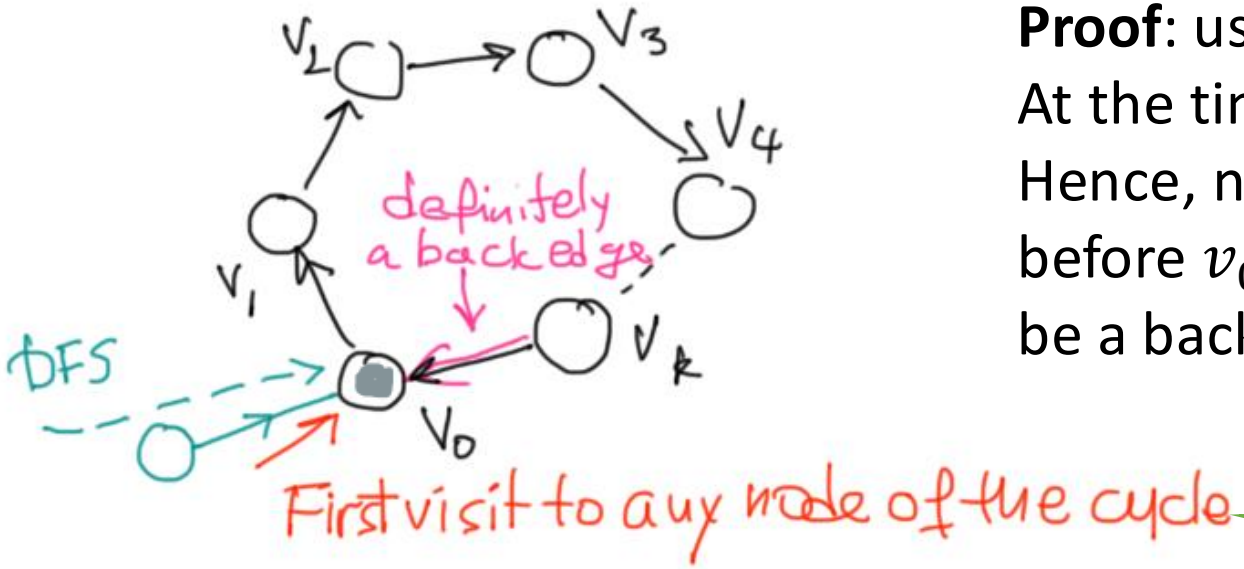
Suppose v is not a descendent of u after the end of the DFS. Let t be the first node along the path *u->v* in G that is not a descendent of u. we show contradiction!
If t not descendent of u, then it was visited after u was finished. But by then all the descendants of *u* were finished, hence also node *s,* and s should have explored the edge (*s,t*), hence *t* should be a descendant of u.

Cycle detection

- **Theorem**: Graph G has a cycle iff DFS has a back edge.
- Proof: a) cycle ⇐ back edge



- b) cycle ⇒ back edge



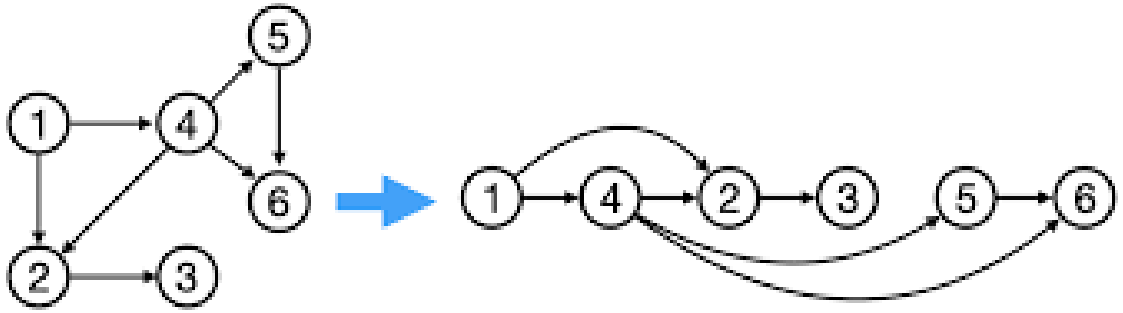
Proof: use the "white path" theorem:
At the time of first visit to v_0, v_k is white.
Hence, node v_k will be finished (become black) before v_0 and it will try the edge (v_k, v_0). It will be a back edge since v_0 is grey at this time.

i.e., assume that v_0 appears on the stack before any other node of the cycle

Topological sort

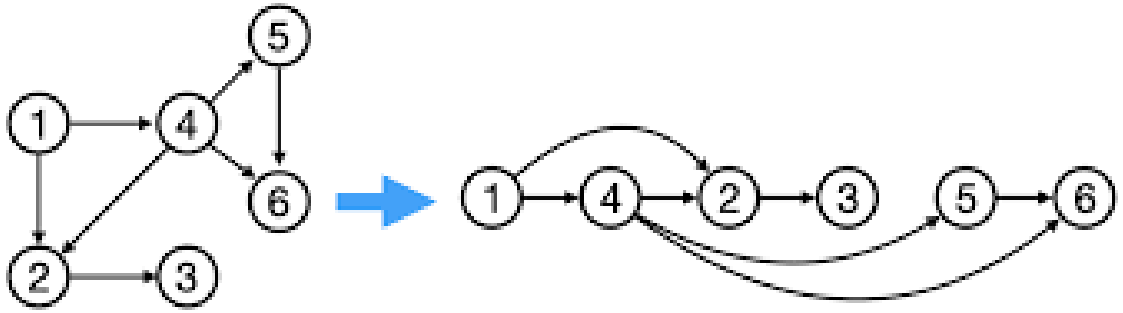
think of an edge $u \to v$ as a "precondition": u must occur before v.

- Topological sort of a DAG $G = (V, E)$.
- **Theorem**: G is a DAG iff there is a ***linear ordering*** of all its $v \in V$ s.t., if $(u, v) \in E$, then $u < v$ in the above ordering, i.e., (u, v) goes always from left to right.



Topological sort

- Topological sort of a DAG $G = (V, E)$.
- **Theorem**: G is a DAG iff there is a ***linear ordering*** of all its $v \in V$ s.t., if $(u, v) \in E$, then $u < v$ in the above ordering, i.e., (u, v) goes always from left to right.



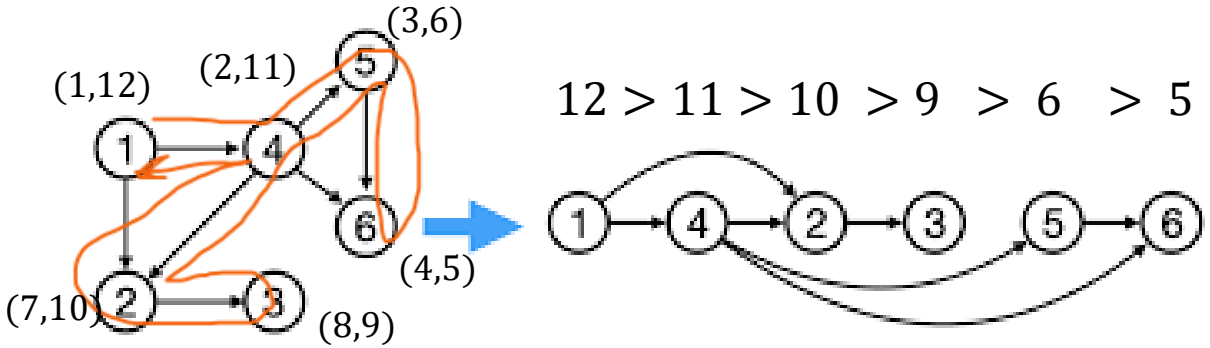
In what sequence did this DFS visit the nodes?

- Algorithm: given DAG G:

1. Run any DFS(G), compute finishing times of nodes.
2. **Linear order**: the nodes in decreasing order of finishing times.

Topological sort

- Topological sort of a DAG $G = (V, E)$.
- **Theorem**: G is a DAG iff there is a ***linear ordering*** of all its $v \in V$ s.t., if $(u, v) \in E$, then $u < v$ in the above ordering, i.e., (u, v) goes always from left to right.



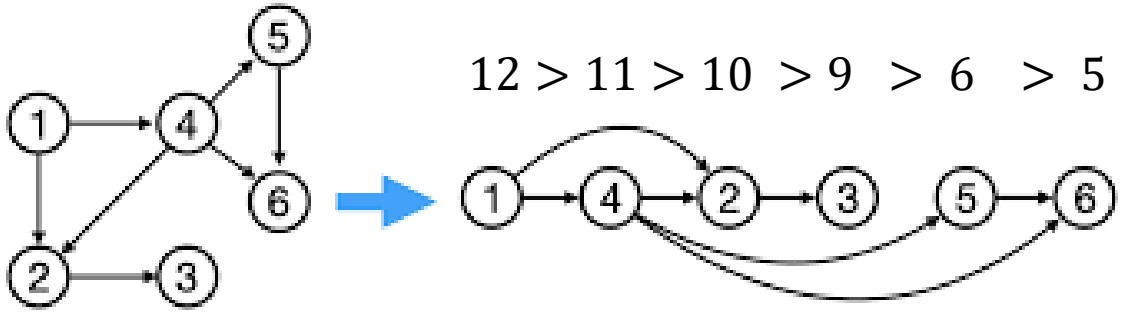
different DFS → perhaps different ordering

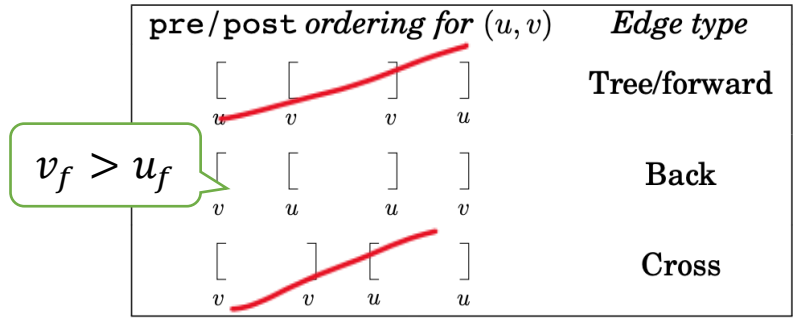
- Algorithm: given DAG G:

1. Run any DFS(G), compute finishing times of nodes.
2. **Linear order**: the nodes in decreasing order of finishing times.

Topological sort

- Topological sort of a DAG $G = (V, E)$.
- **Theorem**: G is a DAG iff there is a ***linear ordering*** of all its $v \in V$ s.t., if $(u, v) \in E$, then $u < v$ in the above ordering.

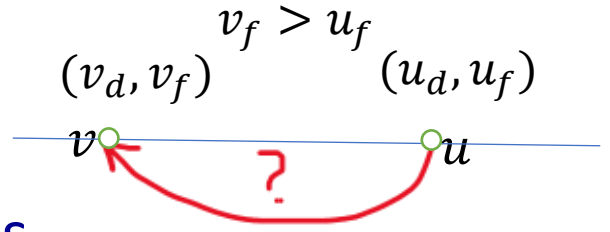




- Algorithm: given DAG G:

1. Run any DFS(G), compute finishing times of nodes.
2. **Linear order**: the nodes in decreasing order of finishing times.

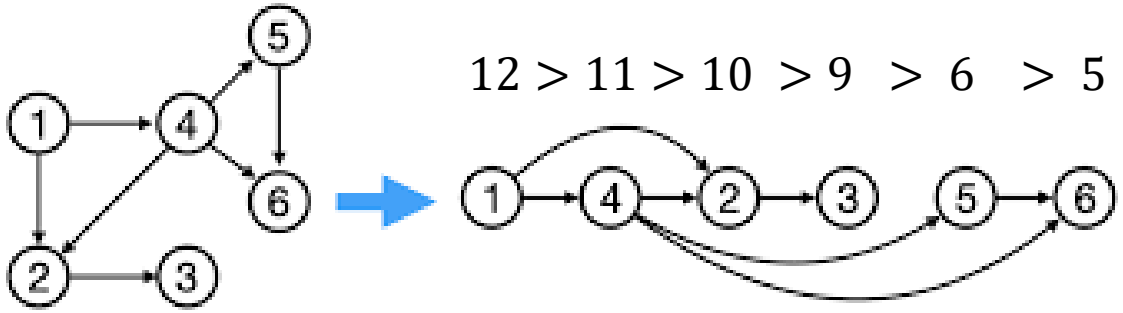
Must prove: if G is a DAG, then there is no edge going right to left.



$u_f < v_f \Rightarrow$ must be back edge, hence cannot be DAG

A corollary

- Topological sort of a DAG $G = (V, E)$.
- **Theorem**: G is a DAG iff there is a ***linear ordering*** of all its $v \in V$ s.t., if $(u, v) \in E$, then $u < v$ in the above ordering.



Corollary: in a DAG, $(u, v) \in E \Rightarrow v_f$ for any depth first search.

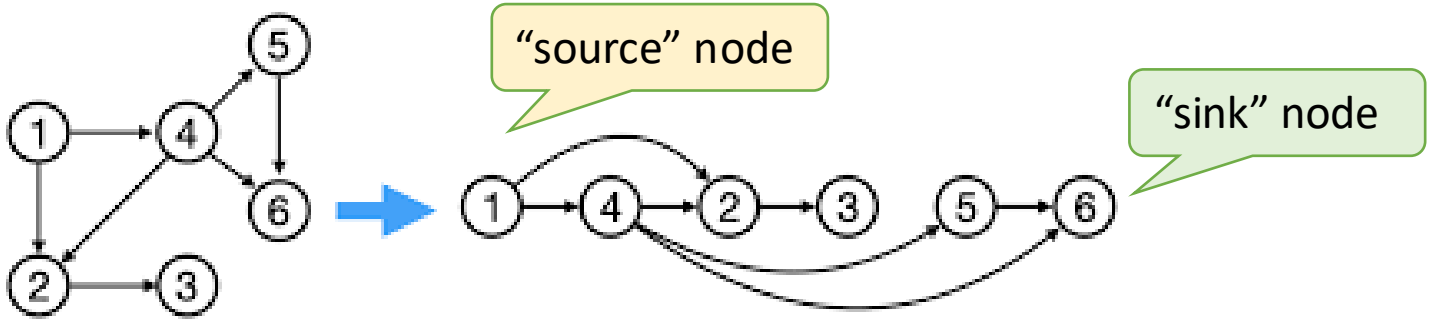
- Algorithm: given DAG G:

1. Run any DFS(G), compute finishing times of nodes.
2. **Linear order**: the nodes in decreasing order of finishing times.

Properties of DAGs

must be a “sink” node

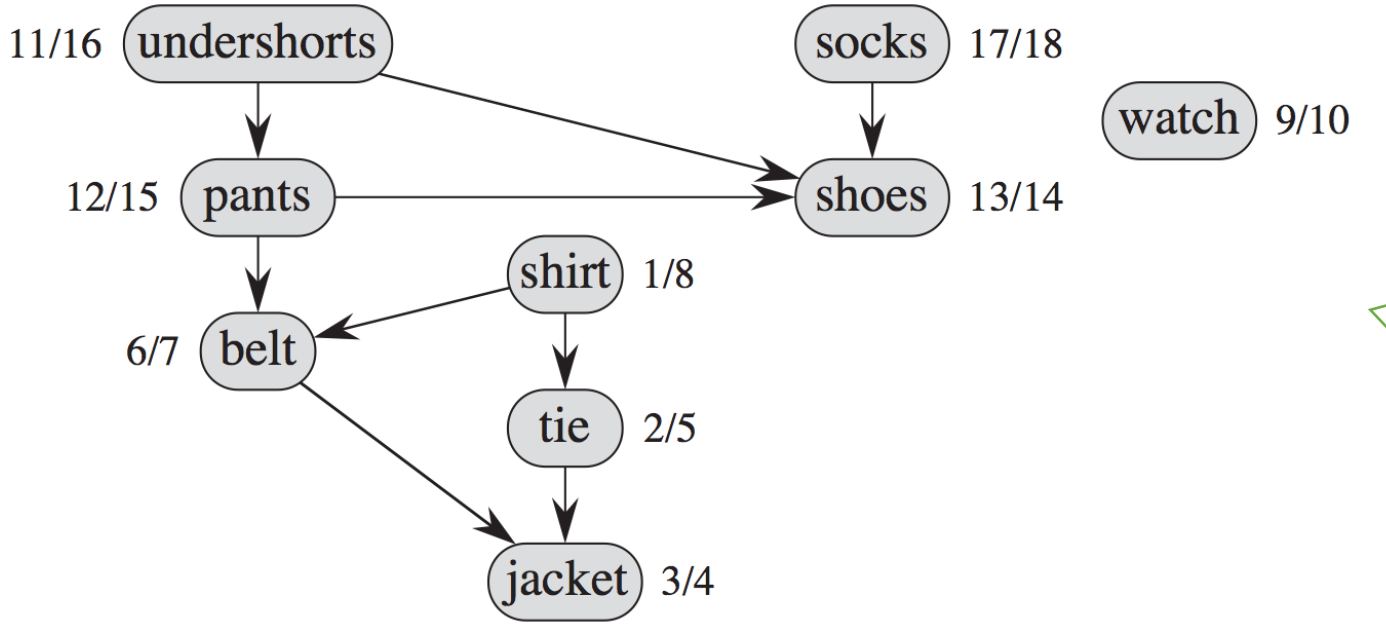
- The last node 6 in the ordering has no outgoing edges.
- If I remove 6 and its incoming edges, then the last node becomes 5, etc.



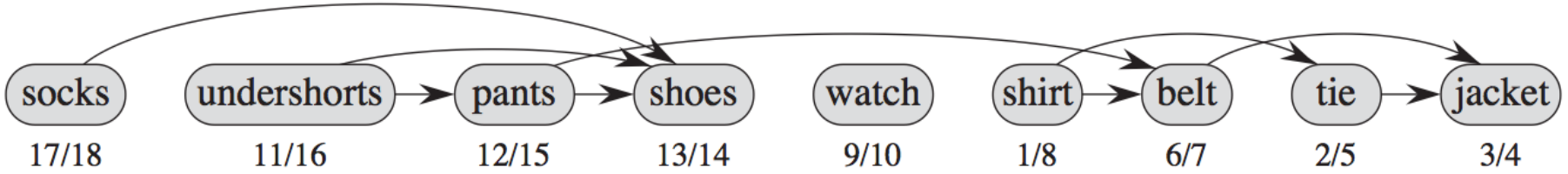
must be a “source” node

- The first node 1 in the ordering has no incoming edges.
- If I remove it, the first node becomes node 4, etc.

Example



directed edge:
action *i* must occur
before action *j*



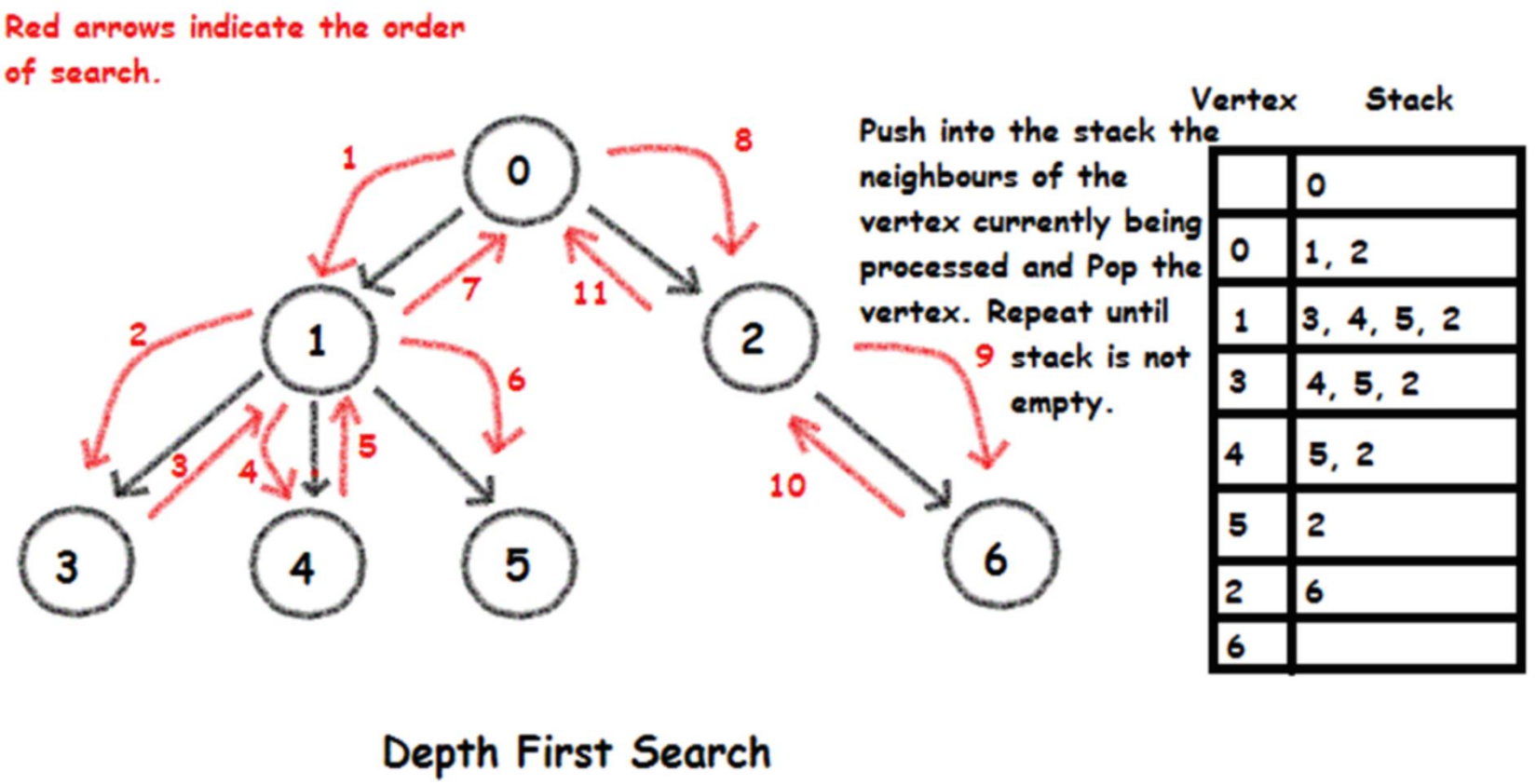
```
GRAPHSEARCH(V, E, s, searchtype)
    // Input: Vertex set V, edge set E, starting vertex s, searchtype as "BFS" or "DFS"
    // Output: Path from s to every vertex in V
    // Running Time: O(|V| + |E|)
 1  for v in V
 2      color[v] ← White
 3      d[v] ← ∞
 4      π[v] ← [] // π[v] is parent of v
 5  color[s] ← Gray; d[s] ← 0; π[s] ← []
 6  if searchtype = "BFS"
 7      Q = queue()
 8  else
 9      Q = stack()
10  Q.push(s)
11  while not Q.empty()
12      u ← Q.pop()
13      for each v in Adj[u]
14          if color[v] = White // v not in Q
15              color[v] ← Gray
16              d[v] ← d[u] + 1; π[v] ← u
17              Q.push(v)
18      color[u] ← Black
```

Non-recursive definition of BFS/DFS

- BFS and DFS are similar except we use a queue for BFS and a stack for DFS (to store the new unexplored nodes).
- BFS: the nodes are explored in sequence they are discovered: **FIFO property of queue.**
- DFS: the nodes are explored in the reverse sequence they are discovered: **LIFO property of stack.**

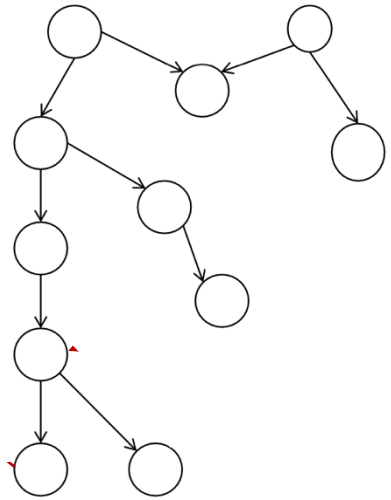
interesting to know that there is a non-recursive definition of DFS

DFS: Stack implementation

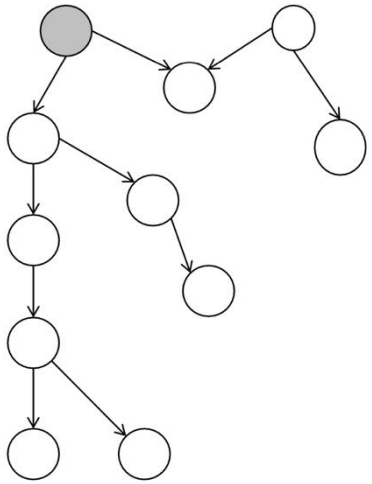


Conclusions

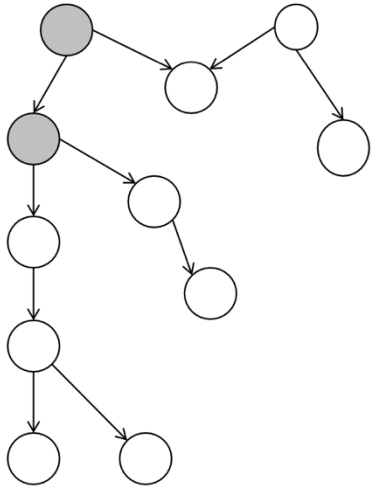
- DFS is a graph search that generates a set of DFS trees.
- It does more than just reachability by characterizing edges of the graph.
  - To characterize edges, it keeps timing information generated during the search.
- Useful to solve problems like cycle detection, topological sort in DAGs, finding strong components in graphs (not examined here), etc.
- DFS can be done as a “dual” of BFS if we use a stack instead of a queue for storing newly discovered nodes.



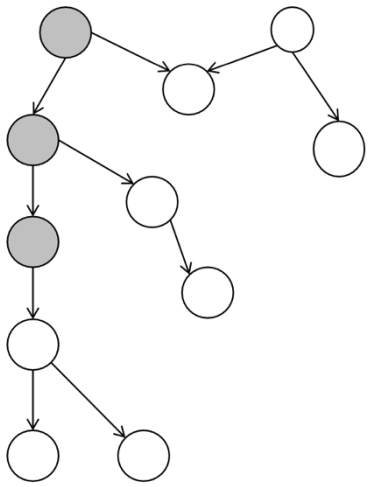
Time=0



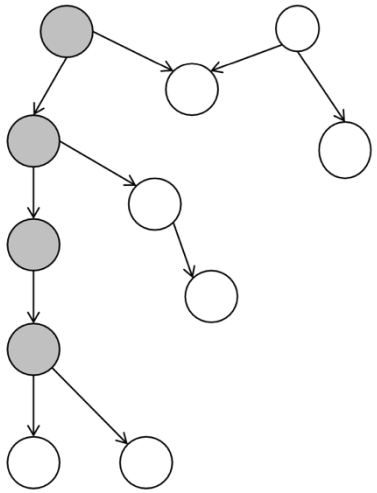
Time=1



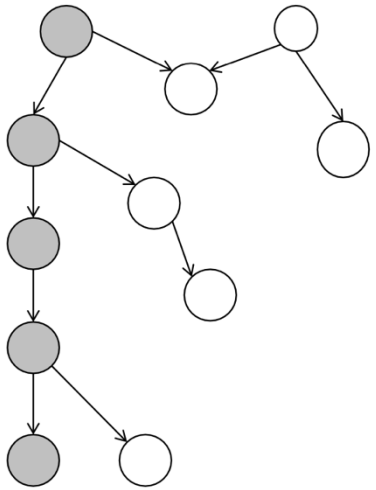
Time=2



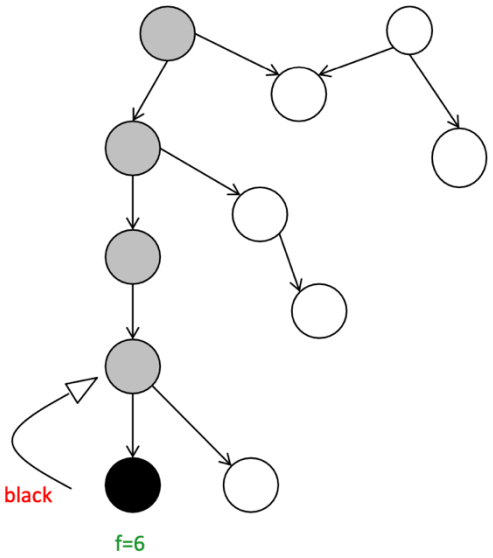
Time=3



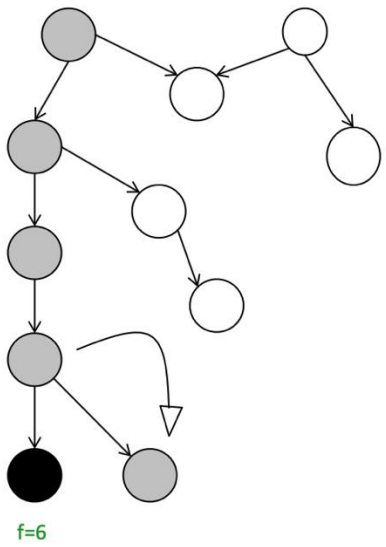
Time=4



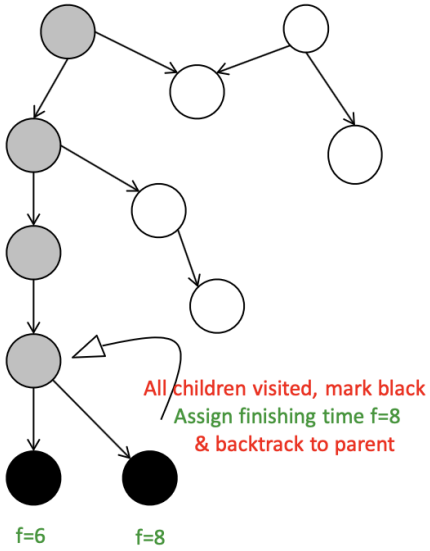
Time=5



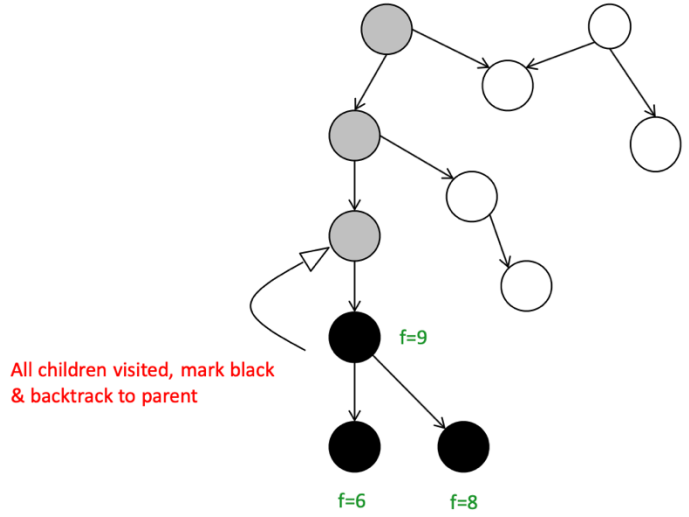
Time=6



Time=7

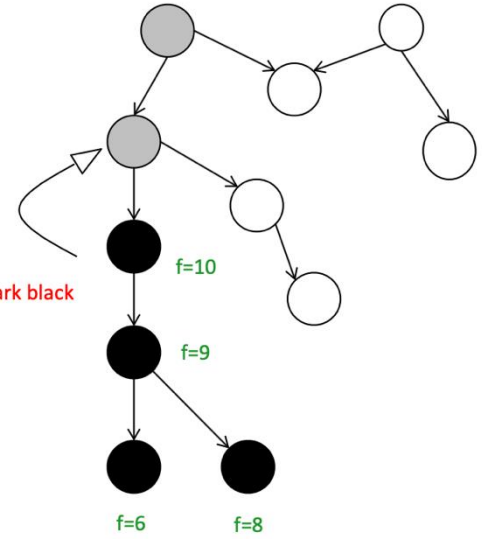


Time=8

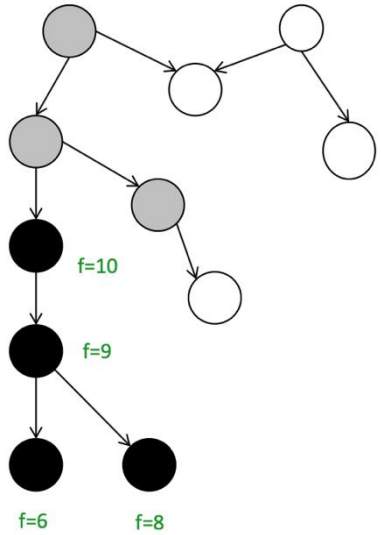


Time=9

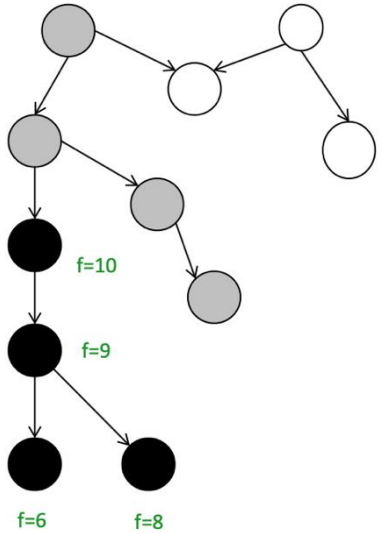
All children visited, mark black & backtrack to parent



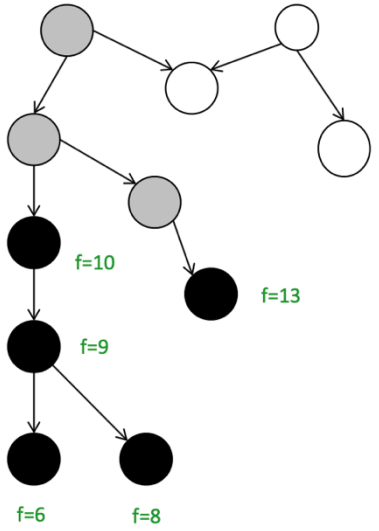
Time=10



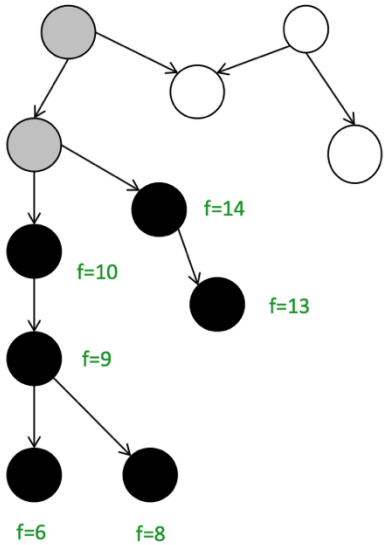
Time=11



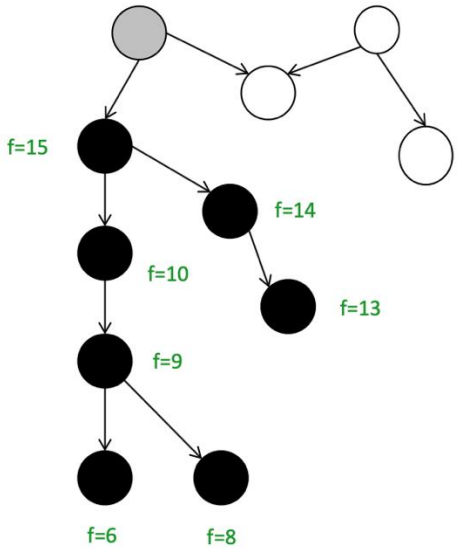
Time=12



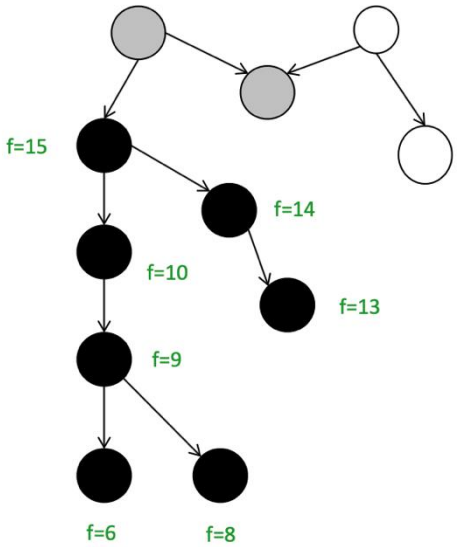
Time=13



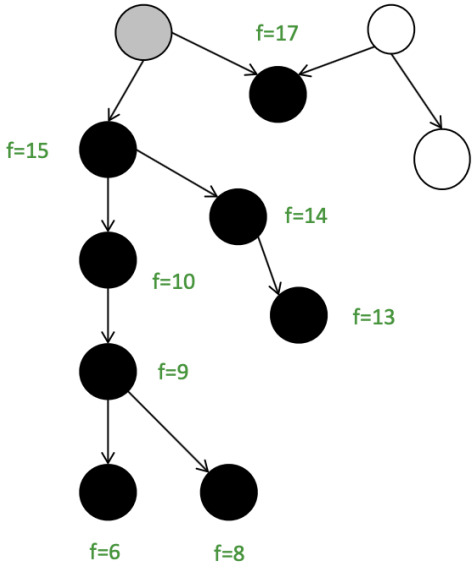
Time=14



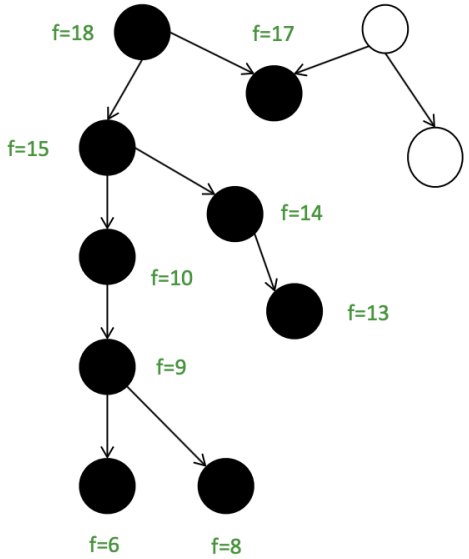
Time=15



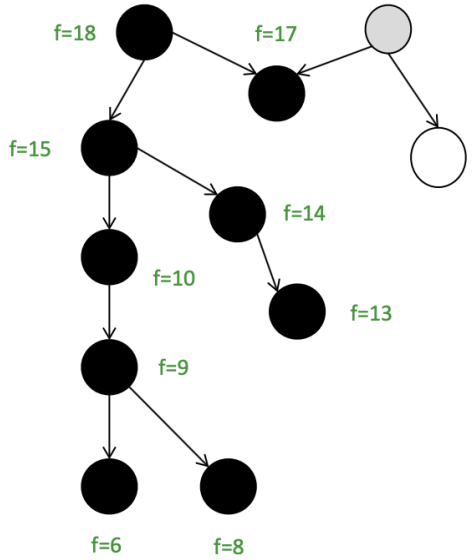
Time=16



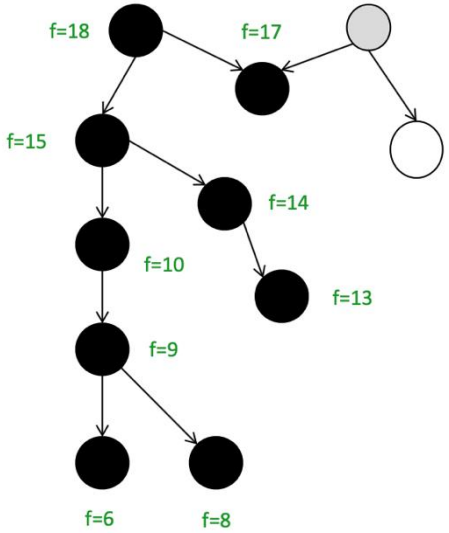
Time=17



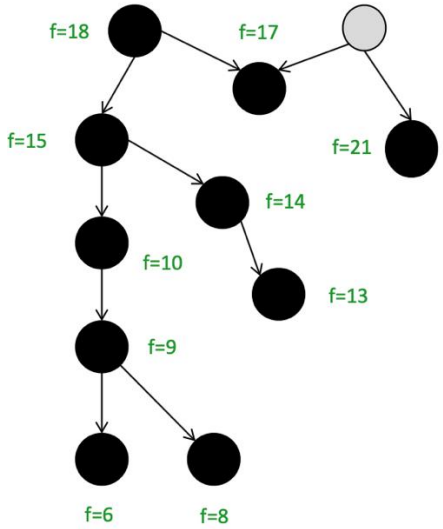
Time=18



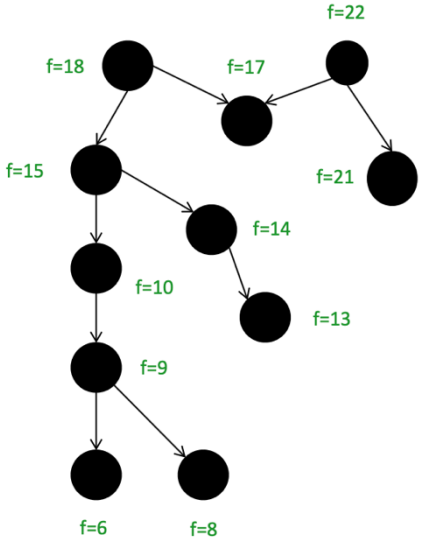
Time=19



Time=19



Time=21



Time=22